

Automated Generation of Decoders for Irregular Instruction Sets Using Information-Theoretic Decision Tree Construction Algorithms

Lillian Tadros

Robotics Research Institute

Technical University of Dortmund

Email: lillian.tadros@tu-dortmund.de

Abstract—Instruction decoders are indispensable components of the System-on-Chip design flow and major constituents of instruction set simulators and processor toolchains. The complex and lengthy process of manual decoder design can be greatly alleviated by automated decoder generation tools based on high level instruction definitions. Unfortunately, automatic generation is challenged by the rising complexity of instruction sets as well as irregularities such as non-uniform opcodes, logic propositions on bit fields and multiple or nested specializations. The few available state-of-the-art decoder generation tools either cannot handle irregularities altogether or produce inadequate results, either functionally or w.r.t. performance. Moreover, they are largely ad hoc and do not bear on any of the well-established work on decision tree generation. This paper presents a sophisticated decision-tree algorithm for the problem of generating decoders for irregular instruction sets. Our algorithm has produced fully automated, functionally correct and cost-aware decoders for the SPARC, MIPS32 and ARMv7 instruction sets. Our results prove the application of information-theoretic concepts to decoder generation a most promising approach.

I. INTRODUCTION

One of the central components in SoC prototyping is the instruction decoder, which is responsible for classifying instructions given the corresponding machine code as defined by an instruction set architecture (ISA). Decoders typically appear as components of different toolchain elements, virtual machines, or instruction-set simulators (ISS). Their range of use covers the whole design cycle, starting with HW/SW co-design, simulation, verification or system support tools. This wide utilization range, coupled with the relative longevity of ISA evolving over time and over microarchitectures sharing the same ISA, makes it highly rewarding to invest in an optimized decoder model that can be repeatedly deployed, modified or expanded.

Manual decoder modeling, however, has reached unprecedented complexity with the recent trend towards more sophisticated ISAs, often encompassing several hundred instructions, numerous instruction formats, multiple sets of extensions (SIMD and others), and sometimes special modes with different instruction lengths (e.g. ARM Thumb and others). To further complicate matter, new instructions are often defined using complex propositions on multiple bit fields, rendering the classical notion of a cleanly defined opcode obsolete, and requiring a completely novel approach towards modeling and manipulating encodings. To date, all current decoder generation algorithms either cannot handle such instruction set irregularities, or claim to do so but fail at corner cases, producing incorrect decoders (see Section II). Surprisingly, they do not at all pertain to the substantial body of work on decision trees, but instead offer rudimentary, use-case-specific solutions. From our point of view, the more complex the ISA, the greater the need for an automated, functionally sound, robust, and optimized decoder generation algorithm.

In this paper, we present an algorithm for generating efficient decoding trees for complex and irregular instruction sets based on information-theoretic concepts. The paper is organized as follows: Section II discusses relevant related work. We proceed with the problem statement and required definitions in Section III. We present and experimentally evaluate our decoding algorithm in Sections IV and V, respectively. We conclude and outline future work in Section VI.

II. RELATED WORK

The initial attempt to achieve fully-automated decision tree generation was made by Theiling [1], which tests all *defined* (i.e. set to either zero or one) bits in the current instruction subset, and assigns the instructions to child nodes according to their mask value. The algorithm does not consider frequencies of instruction occurrence or tree cost, which can, in the extreme case, degenerate into a shallow but correspondingly wide table with many memory-consuming branches. Most importantly, the algorithm can neither handle non-uniform ISAs nor propositions on instruction fields (see Section III-B), and would thus fail on ISAs such as ARM (version $\geq v6$) or MIPS (version $\geq R6$). Nevertheless, the algorithm initiated the study of fully automatic decoder generation and was reproduced in other work [2].

The PART algorithm by Fournel et al. [3] expands Theiling's work to handle logic propositions. The authors define propositions in terms of binary decision diagrams (BDD), which are then expanded to all valid encodings. The methodology retains all of the original algorithm's shortcomings, as well as adding a highly inefficient preprocessing step which involves exponential expansion of the instruction set.

Later work by Okuda et al. [4] also builds upon Theiling's work by endeavoring to handle non-uniform ISAs and propositions. In contrast to Fournel, propositions are treated as first-class entities and are included as possible splitting criteria. The flaws with respect to performance are retained from the original algorithm and are even exacerbated w.r.t. functionality: The formalism for instruction definition lacks the expressiveness required by many modern ISAs (including even the aesthetic and spartan MIPS), such as inequalities or relations between instruction fields. Furthermore, the algorithm attempts to handle non-uniform ISAs by resorting to propositions, which is not guaranteed to provide a solution or, even worse, may even create functionally erroneous decoders.

Qin et al. [5] addresses some of the deficiencies in Theiling's algorithm by handling non-uniform ISAs, allowing splitting on undefined bits, assigning a heuristic to memory consumption to limit the number of branches, and addressing probabilities by estimating the height of a best-case Huffman subtree. The search space is reduced by imposing contiguity on the chosen masks. Notwithstanding, the algorithm is markedly more compute-intensive than Theiling. Surprisingly, as the measurements in [6] show, these attempts at optimality fail in improving the performance. On the contrary, Qin's algorithm performs slightly worse. As discussed in this survey work, this could well be due to the restriction on mask contiguity, which probably leads to excluding more efficient decoders, as well as to the adopted cost model, which is shown to be irrelevant in regard to the real performance [7]. Likewise, the heuristic for memory consumption is highly disputable.

The EFF algorithm by Fournel [3] attempts to handle propositions by applying Qin's algorithm to a BDD representation of the instruction. By definition, however, Qin's bitwise construction of the decision function must necessarily fail on all propositions larger

than two bits. The authors vaguely mention “helping” the decoder with “special pattern[s]”, i.e. presumably hardcoding all propositions which the algorithm would not find otherwise.

Later work by Wagstaff [8] falls back to non-optimal bitwise mask construction. An algorithm by Solovev [9] does not rely on detailed instruction definitions, but rather attempts to reconstruct the instruction encodings from a combination of generic format descriptions, assembly and machine code, which necessarily requires large training data.

Though [10] offers functionally sound decoders for non-uniform ISAs with complex conditionals, it remains ad-hoc and somewhat haphazard. With the non-optimized solution showing better performance than the optimized, there clearly remains a need for a more rigorous approach.

In summary, the inadequacy of existing automatic decoder generators is perhaps best attested by the following two indicators: First, the ongoing research on refinements of and alternatives to interpreted simulation, including caching, static, hybrid, just-in-time and dynamic compilation [11], which, nonetheless, fail to provide the level of both flexibility and accuracy offered by interpreted simulation. Second, after decades of research invested in automatic decoder generation, manual decoders remain as extant, relevant, and ubiquitous as ever [12]–[14].

III. BACKGROUND AND DEFINITIONS

A. Instructions, Encodings and Decision Trees

An *instruction instance* or bit string s is a string over the binary alphabet. An *instruction encoding* p is a string over the ternary-logic alphabet, which includes *defined* (zero or one) as well as *undefined* or “don’t-care” bits. Bit extraction, slicing, and concatenation in most-significant-bit (MSB) order are denoted by $p[i]$, $p[i:j]$, and $p[i]:p[j]$, respectively. An ISA defines a set E of *encoding entries* e , which assign a *label* l to each encoding. With the aid of benchmarking, an encoding entry can be extended with a *probability of occurrence* o based on how often instructions pertaining to this encoding are encountered.

The purpose of a *decision tree* is to relate a given instruction instance to the correct encoding. This process is called *matching* and denoted $s \in p$. Internal nodes contain *decision functions* f , which direct the instruction in question to the correct branch. The decision tree leaves are tagged with instruction labels. These definitions assume a *well-formed* ISA, i.e. where each string can only match one encoding.

B. Properties of Instruction Sets

TABLE I: Example irregular instruction set.

	ENCODING	FEATURE
A	[= 11-- 1---]	Non-uniform opcodes (A, B, C, D)
B	[= 0-1- ---0]	Non-identification bit $B[0]$
C	[= -00- 0---]	
D	[= 10-- 1---] $\wedge [\sim \text{----} -111]$	Boolean proposition
AA	[= 11-- 111-]	Specialization of A
AB	[= 11-- 10-1]	Another specialization of A
AAA	[= 11-- 1111]	Nested specialization of A via AA
E	[= 0101]	Non-uniform instruction length

Table I. The instruction encodings are defined using a notation where the bits are listed in MSB order, defined bits are written as ‘1’ or ‘0’, and undefined bits as ‘-’. Instruction D contains an exclusion condition: The three least-significant bits cannot equal 111₂. See note on notation in Section IV-A and Table II.

Table I illustrates some of the challenging features of modern instruction sets, such as *non-identification bits*, *specializations* and *non-uniform instruction sizes*. While these three are relatively simple

to address, the following two properties are more involved and remain poorly handled by current generator tools:

a) *Non-uniform opcodes*: An opcode is the smallest set of bits that are sufficient to uniquely identify an instruction. Traditionally, opcodes consisted of the same adjacent set of bits in all instructions. Thus, simply masking the opcode field was enough for classifying instructions. In modern instruction sets this is hardly the case. In order to expand the ISA while retaining some backward compatibility, existing encoding formats are often expanded with sub-opcodes, special opcodes, and so on. This has two effects on decoding: First, the number of bits required for identifying instructions generally grows. In the worst case, it can degenerate to all n instruction bits, instead of the lower limit of $\lceil \log_2(m) \rceil$ that are strictly required to classify m encodings. In Table I, decoding instructions A, B, C and D requires checking bits 3, 5, 6 and 7. Second, since different formats define opcodes anywhere over the whole instruction, leaving others undefined, it can often occur that there is no single bit that is defined for all encoding formats. This is the case for instructions A, B, C and D in Table I, which do not share a single common defined bit. If an algorithm can only test defined bits, it will fail on valid ISAs, as does Theiling’s algorithm discussed above. Other algorithms solve such cases of non-uniform opcodes by *duplication*: If some encodings do not yield a unique result of the chosen decision function, they are assigned to all matching branches.

b) *Propositions*: Traditional encoding definitions are simple conjunctions of bits. More recent ISAs make use of propositions, which entail more complex restrictions on bit fields. The simplest propositions are negated conjunctions, such as in instruction D in Table I. In general, propositions lead to rejecting some strings that would otherwise match an encoding. As with other parts of an encoding, propositions can either be required for identification or at least for correctness. The proposition on instruction D pertains to the latter case.

Although we chose to analyze three RISC ISAs, none of them exhibits trivial encoding complexity. The simplest was SPARC [15], which avoided propositions and non-uniform opcodes, but still made use of other complex features. The other two, ARMv7 [16] (including floating-point and SIMD instructions), and MIPS R6 [17] boast almost all of the above-mentioned properties.

IV. DECODER CONSTRUCTION

A. Formalizing Encodings

The definitions provided so far are bit-oriented and not tailored to handle propositions. The propositions in both ARMv7 and MIPS32 can involve either equalities, *disequalities* or inequalities, all of which can be defined either between a field and a constant or between two fields. We propose a formalism which we illustrate in Table II. Each square bracket is a predicate that applies the operator inside the brackets to the indicated bits. An operation between two fields is represented using letters to mark each field. The first line typically corresponds to a “classical” encoding definition, i.e. a conjunction of bits.

It is easy to see that our representation is a shorthand for first-order (predicate) logic statements in conjunctive normal form, without quantification and with predicates in the binary domain. As such, each encoding is written as a conjunction of disjunctions of predicates. The MIPS BEQZALC encoding pattern can thus be rewritten as $\neg p[31] \wedge \neg p[30] \wedge p[29] \wedge \neg p[28] \wedge \neg p[27] \wedge \neg p[26] \wedge (p[20:16] \neq 0000_2) \wedge (p[25:21] \geq p[20:16])$. We formalize our representation by the following BNF:

```

encoding ::= clause (  $\wedge$  clause ) *
clause   ::= predicate
           | ( ( predicate (  $\vee$  predicate ) + ) )
predicate ::= [ op literal-or-field + ]
op         ::= = | ~ | < | > | <= | >=
literal-or-field ::= 0 | 1 | - | A | B | ...

```

TABLE II: Examples encodings in ARMv7 and MIPS32 R6.

LABEL	ENCODING
ARMv7 ADD (imm)	[= ---- 0010 100- ---- - - - - - - - -]
	^ [~ 1111 ---- - - - - - - - - - - - -]
	^ [~ ---- - - - - - - - - 1101 ---- - - - -]
	^ [~ ---- - - - - - - - - 0 1111 ---- - - - -]
	^ [~ ---- - - - - - - - - 1 ---- 1111 ---- - - - -]
ARMv7 RSB (reg)	[= ---- 0000 011- ---- - - - - - 0 ----]
	^ [~ 1111 ---- - - - - - - - - - - - -]
	^ ([= ---- - - - - - - - - 0 ---- - - - - - -]
	^ [~ ---- - - - - - - - - 1111 ---- - - - -])
MIPS32 BEQZALC	[= 001000 ---- - - - - - - - - - - - -]
	^ [~ ---- - - - - 00000 ---- - - - - - -]
	^ [>=----- SSSSS TTTT ---- - - - - - -]

Table II. The table lists some encodings from ARMv7 and MIPS32 R6 using the proposed notation in MSB order. Each predicate applies the operator inside the bracket to the encoding and the bracket value. The second line can thus be interpreted as $p[31:28] \neq 1111_2$. In the case of predicates between two fields, each field is designated using a letter, and assigned to the operator in reading order. The last line is e.g. $p[25:21] \geq p[20:16]$. The first line in each encoding is usually a simple conjunction of bits and corresponds to “classical”, proposition-free encodings.

Our formalism suffices to describe the full ISA of our three platforms. The primary advantage is that there is generally no exponential expansion of predicates involved, which would be the case if we attempted a conversion to zeroth-order (propositional) logic. Furthermore, not requiring bit field contiguity generally simplifies the notation. Most importantly, the notation abstracts predicates from their bit constituents and makes them first-class citizens that are on equal footing with single bits. As will be apparent presently, this opens up new possibilities in decision tree construction.

B. Decision Functions

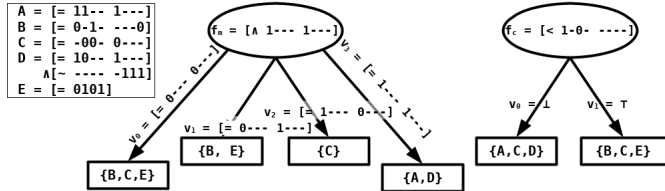


Fig. 1: Two example decision functions applied to the instruction set in Table I, where subinstructions are automatically assigned to all branches containing the parent and therefore not explicitly handled. N-ary functions such as f_m can only apply one type of operator: the ternary-logic conjunction operator $\wedge$, and result in at most 2^J branches for $|J| = \text{defined}(f_m)$. The values are attached to the branches and specify the results of applying the decision function. Binary functions such as f_c can apply any relational operator (in this case: less-than), and result in two branches with boolean truth values. Our model of computation interprets defined bits in MSB order and ignores undefined ones, so that f_c is interpreted as $p[7]:p[5] \leq 10_2$.

We define two types of decision functions:

a) *Mask functions*: f_m are n-ary functions that first select a set of instruction bits via a given mask, then assign the instruction to the matching branch depending on the result. Formally, they apply some string over the alphabet $\{1, -\}^n$ using the conjunction operator $\wedge$. A mask where $|\text{defined}(f_m)| = J$ results in 2^J values $v_j \in \{0, 1, -\}^n$, which are used to tag the corresponding child branches. The values v_j are written using a restricted form of the encoding formalism of the previous section, which is a single equality predicate of the form $[= \dots]$.

b) *Condition functions*: f_c are binary functions that compare the instruction instance to a fixed value using relational operators such as equality, disequality or inequality, resulting in two (matching and unmatching) branches, tagged with $v_1 = \top$ and $v_0 = \perp$, respectively. Compared to masks, binary functions are useful when it is more efficient to split exactly one instruction away from the set. They can be regarded as a specialization of masks, where one branch value is

kept (matching) and all others subsumed in the *unmatching* branch. A second advantage is that, unlike bit-oriented masks, they can handle conditions. As mentioned in the previous section, we treat both bits and conditions as atoms. In fact, each predicate contained within a clause, within a term, is treated as an atom, and is thus considered a possible splitting function. To be as general as possible, we allow condition functions to take any form that complies to the complete grammar of Section IV-A. This enables us to split off any instruction from the current set by simply setting its complete encoding as the decision function.

Fig. 1 illustrates both function types. Either type utilizes three-bit logic, meaning that the functions can operate on a not necessarily contiguous subset of the instruction bits. The definition of matching will be expanded shortly, but, in general, we say that an instruction matches a *binary function*, but matches an *n-ary function value* as assigned to a given branch.

C. Matching

Matching is relevant to three different contexts: During decoding, an instruction string is successively matched against each decision function ($s \in f$), until it is finally matched to an encoding ($s \in p$). During *decoder generation*, we start with a set of encodings at the root, which is iteratively split by matching an encoding against a decision function ($p \in f$). This generic definition of matching is made possible by our formalism, which models instructions, encodings and decision functions alike. Without loss of generality, we concentrate on the third context in the following discussion, as it is the most complex case and the one most relevant to decoder generation. An initial, bit-based definition of matching might look thus:

$$\begin{aligned} \text{Full match } p \in f: & \quad \forall i \in [0, n-1] \quad p[i] = f[i] \quad \vee \quad f[i] = - \\ \text{Partial match } p \in f: & \quad (\forall i \in [0, n-1] \quad p[i] = f[i] \quad \vee \quad f[i] = - \quad \vee \quad p[i] = -) \\ & \quad \wedge \quad (\exists i \quad p[i] = - \quad \wedge \quad f[i] \neq -) \\ \text{Unmatch } p \notin f: & \quad \neg(p \in f) \quad \wedge \quad \neg(p \in f) \end{aligned}$$

These definitions are for binary functions. For mask functions, $p \in v_j$, $p \in v_j$ and $p \notin v_j$ are defined accordingly. These definitions cannot handle conditions. The examples below illustrate how the interplay of bits and conditions can affect the matching result for both decision function types. As shown, conditions can change a partial or a full match one into the other, or either to an *unmatch*:

$$\begin{aligned} f_m &= [= 11-- 11--], v_1 = [= 00-- 01--] \\ p_1 &= [= 00-- ----] \wedge [> ---- 01--] \models p_1 \notin v_1 \\ p_2 &= [= 00-- 0---] \wedge [> ---- -010] \wedge [\sim ---- -11] \\ &\models p_2 \in v_1 \\ f_{c1} &= [< ---- 10--] \\ p_3 &= [= 00-- 1---] \models p_3 \notin f_{c1} \\ p_4 &= [= 00-- ----] \wedge [> ---- 01--] \models p_4 \notin f_{c1} \\ p_5 &= [= 00-- ----] \wedge [\sim ---- 1---] \models p_5 \in f_{c1} \\ p_6 &= [= 00-- 0---] \wedge [\sim ---- -1--] \models p_6 \in f_{c1} \\ f_{c2} &= [= 00-- ----] \wedge [\sim ---- 00--] \\ p_7 &= [= 00-- ----] \wedge [< ---- 11--] \models p_7 \in f_{c2} \\ p_8 &= [= 00-- ----] \wedge [< ---- 01--] \models p_8 \notin f_{c2} \end{aligned}$$

We address such cases by a new definition of matching that is in line with our formalism of IV-A. Informally, a full match is when all clauses set by the decision function are fulfilled, i.e. implied by, the encoding. An *unmatch* is when the encoding contradicts the decision function. And a partial match is when neither case is true. This can be formalized using either logic- or set-theoretic concepts:

$$\begin{aligned} \text{Full match } p \in f: & \quad p \rightarrow f & f &\subseteq p \\ \text{Partial match } p \in f: & \quad \exists x(x \in p \wedge x \in f) \wedge \exists y(y \in p \wedge y \notin f) & f &\not\subseteq p \wedge p \cup f \neq \phi \\ \text{Unmatch } p \notin f: & \quad p \rightarrow \neg f & f &\subseteq p' \end{aligned}$$

D. Base Algorithm

Building the decoding tree is a recursive process described in pseudocode in Algorithm 1.

Lines 2-4 are the terminal condition of the recursion, where the set of encoding entries contains only one entry p_0 . The node is thus labeled with the encoding name or identifier.

Algorithm 1 Base Algorithm

```

    ▷ Remove substr, from encoding entry set  $E$  and attach to parents.
1: function MAKETREE( $node, E$ )
    ▷ Terminal Condition:
2:   if  $|E| = 1$  then
3:     if  $e_0.encoding \subseteq node.cum$  then
4:        $node.label \leftarrow e_0.label$ 
    ▷ Create final node with all remaining untested bits and conditions:
5:   else
6:      $node.func \leftarrow e_0.encoding - node.cum$ 
7:      $child \leftarrow CreateNode(\{e_0\})$ 
8:      $child.cum \leftarrow CumFunc(node.cum, node.func, v=1)$ 
9:      $node.AddChild(child, v=1)$ 
10:     $MakeTree(child, \{e_0\})$ 
11:   end if
12:   return
13: end if
    ▷ Recursive Case:
14:    $(f, \{(child_j, v_j, E_j) \mid 0 \leq j < J\}) \leftarrow SelectFunction(node, E)$ 
15:    $node.func \leftarrow f$ 
16:   for  $(child_j, v_j, E_j) \in \{(child_j, v_j, E_j) \mid 0 \leq j < J\}$  do
17:      $child_j.cum \leftarrow CumFunc(node.cum, f, v_j)$ 
18:     for  $e \in \{E_j\}$  do
19:        $e \leftarrow Simplify(f, v_j, e)$ 
20:     end for
21:      $node.AddChild(child_j, v_j)$ 
22:      $MakeTree(child_j, E_j)$ 
23:   end for
24: end function

```

Lines 6-10 specify the last-but-one iteration: To test non-identification clauses (Section III-B), we create an additional node containing any remaining untested bits or conditions.

The functions *CreateNode* and *AddChild* are related to the tree structure and thus not relevant to the discussion. The functions *CumFunc* and *Simplify* that build the cumulative decision function and simplify the encodings, respectively, will be described in Section IV-E. The lines 14-23 describe the recursive case. The *SelectFunction* is the heart of the algorithm, the details of which we will discuss in Section IV-F. The procedure examines several decision functions, performs the matching according to Section IV-C, evaluates the functions, then returns the selected function f along with a set of children. Each child node $child_j$ is linked to a value v_j that labels the corresponding branch, as well as to its set of matching encodings E_j .

E. Post-Matching

We need to keep track of the sequence of tested decision functions for two reasons: First, to ensure that each encoding is fully tested. Second, during decoder generation, the set of eligible decision functions can be reduced by subtracting all the previously tested. For masks, we simply store the conjunction of all tested bits. Condition functions bits, on the other hand, cannot, in general, be considered tested, except for the matching branch. The *unmatching* branch of e.g. $f_c = [1111 \text{ ---}]$ only excludes one possibility out of 16, and thus requires further testing of the same bits. The only case when the *unmatching* branch is considered tested is when the function involves only one bit. In general, if the function application yields true for exactly one combination of bit values, then these bits are considered tested and added to the cumulative decision function.

After matching, all child encodings are examined and simplified against the decision function. The following MIPS32 encodings illustrate why this is necessary:

BOVC	=	[= 001000 ---- - - - - -]
	$\wedge$	[>=----- xxxxx yyyyy - - - - -]
BEQZALC	=	[= 001000 00000 - - - - -]
	$\wedge$	[~ ----- 00000 - - - - -]
f_c	=	[= ----- 00000 - - - - -]
$v_1 = T$,		BOVC, BEQZALC $\in v_1$:
BOVC	=	[= 001000 00000 00000 - - - - -]
BEQZALC	=	[= 001000 00000 - - - - -]
	$\wedge$	[~ ----- - - - - - 00000 - - - - -]

It is easy to see that, if *BOVC* is *not* simplified after applying f_c , the two encodings would be indistinguishable. Generally, simplification involves two aspects: Writing out bits that are set by the decision function but undefined by the encoding, and/or simplifying encoding conditions. The latter step could, in turn, simplify the condition up to the point where we can write out the condition as set bits. Furthermore, if any newly written bits intersect with any conditions, the process needs to be repeated. Fig. 2 illustrates several cases.

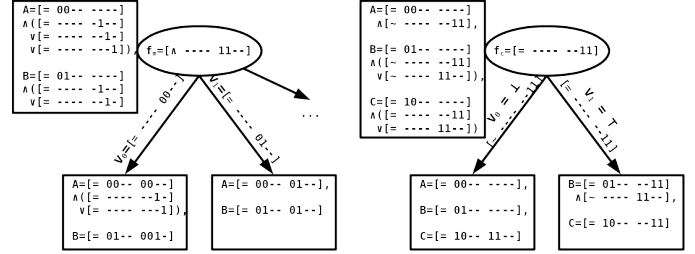


Fig. 2: Simplifying Encodings: Two decision functions applied to the example instruction sets in the top boxes. Both functions involve writing bits as well as simplifying conditions. In general, a false predicate is removed from the enclosing clause (e.g. $(f_c, v_1)(B)$), while a true predicate causes the enclosing clause to be removed (e.g. $(f_c, v_0)(B)$). Simplified clauses can be written out as bits if possible (e.g. $(f_c, v_0)(C)$).

F. Evaluating Decision Functions

Since our aim is to construct efficient decision trees, we need a metric to quantitatively assess decision functions. We agree with the established notion that takes average execution speed as the major cost indicator [5]. An exhaustive solution that generates all possible trees and benchmarks their execution is obviously not feasible. Decision tree literature (e.g. [18]) present many different criteria for top-down evaluation and selection of decision functions at the local level. However, not all of these are applicable or relevant to our use case, so we concentrate on four criteria that especially target classification efficiency and assess their applicability to our use case.

Table III illustrates this calculation stepwise for the example instruction set shown in the upper part of the table. The next part calculates contingency values, which denote the occurrence of a given dependent variable (in decision tree jargon: *class* c) in the presence of a given independent variable (*predictor* or *attribute* a). In our case, the classes are the encoding labels, and the predictors are either single bits or single predicates set to a given attribute-specific value (*category* or *value* $v \in V_a$). Either predictor type results in a binary split according to the matching principles of Section IV-C. A bit predictor that is defined in the encoding leads to the values 1 (full match) and 0 (*unmatch*), depending on the bit value in the encoding. For example, the encoding $A = 1-11$ has the value 0 for predictor $p[3] = 0$ ($f_m = [1-1-]$, $v_0 = [0-1-]$), and the value 1 for predictor $p[3] = 1$ ($f_m = [1-1-]$, $v_1 = [1-1-]$). This can be interpreted thus: Should the encoding set be split by the predictor $p[3]$, encoding A would only be assigned to the matching branch. A partial match due to an undefined bit leads to contingency of 0.5, as the encoding is equally likely to occur in either branch. Care must be taken in the presence of conditions. Consider encoding $D = [1-0-] \wedge ([1-1-] \vee [1-0-])$. Writing out the conditions yields $D = [1-00] \vee [1-10] \vee [1-11]$. The predictor $p[3]$, e.g., would then result in D partially matching v_0 and v_1 with a one-third and two-thirds probability, respectively.

The contingency values are then used to calculate the following four metrics. The third part of Table III shows the calculation results for an excerpt of the example instruction set.

a) *Chi-Squared*: Chi-squared is a classical metric used e.g. in [19] and others. We first calculate the expected value E followed by χ^2 for a given value v and class c combination, then build the sum over all classes and all attribute values. Predictors with a higher χ^2 value are considered better splitting candidates:

TABLE III: Contingency table and decision function metrics for an example instruction set.

CLASS	ENCODING	PATTERNS	$p(c)$
A	[= 1-11]	[= 1-11]	0.41
B	[= 0---] $\wedge$ [~ --00] $\wedge$ [~ --11]	[= 0-01] $\vee$ [= 0-10]	0.28
C	[= --1-] $\wedge$ [~ 0--0] $\wedge$ [~ 1--1]	[= 0-11] $\vee$ [= 1-10]	0.19
D	[= --0-] $\wedge$ ([= 1---] $\vee$ [= ---0])	[= 0-00] $\vee$ [= 1-00] $\vee$ [= 1-01]	0.12

BIT CONTINGENCY TABLE ($p_v(c)$)									
Class	b[3]=0	b[3]=1	b[2]=0	b[2]=1	b[1]=0	b[1]=1	b[0]=0	b[0]=1	$p(c)$
A	0-0.41	1-0.41	0.5-0.41	0.5-0.41	0-0.41	1-0.41	0-0.41	1-0.41	0.41
B	1-0.28	0-0.28	0.5-0.28	0.5-0.28	0.5-0.28	0.5-0.28	0.5-0.28	0.5-0.28	0.28
C	0.5-0.19	0.5-0.19	0.5-0.19	0.5-0.19	0-0.19	1-0.19	0.5-0.19	0.5-0.19	0.19
D	0.33-0.12	0.57-0.12	0.5-0.12	0.5-0.12	1-0.12	0-0.12	0.57-0.12	0.33-0.12	0.12
$p(v)$	0.415	0.585	0.5	0.5	0.26	0.74	0.315	0.685	1

ATTRIBUTE METRICS					
Chi-squared:	$\chi'^2(a) = \sum_{v \in V_a} p(v) \cdot \sum_{c \in C} \left(\frac{(p_v(c) - p(c))^2}{p(c)} \right)$				
$\chi'^2(b[1]:0)$	$= \frac{(0.0.41 - 0.41)^2}{0.26} + \frac{(0.5-0.28 - 0.28)^2}{0.26} + \frac{(0.0.19 - 0.19)^2}{0.26} + \frac{(1-0.12 - 0.12)^2}{0.26} \approx 1.81$				
$\chi'^2(b[1]:1)$	$= \frac{(0.0.41 - 0.41)^2}{0.74} + \frac{(0.5-0.28 - 0.28)^2}{0.74} + \frac{(1-0.19 - 0.19)^2}{0.74} + \frac{(0.0.12 - 0.12)^2}{0.74} \approx 0.224$				
$\chi'^2(b[1])$	$\approx 0.26 \cdot 1.81 + 0.74 \cdot 0.224 \approx 0.636$				
$\chi'^2(a)$	0.695	0	0.636	0.332	
Gini Index:	$Gini(a) = \sum_{v \in V_a} p(v) \cdot (1 - \sum_{c \in C} p_v(c)^2)$				
$Gini(b[1]:0)$	$= 1 - \left(\frac{0.0.41}{0.26} \right)^2 - \left(\frac{0.5-0.28}{0.26} \right)^2 - \left(\frac{0.0.19}{0.26} \right)^2 - \left(\frac{1-0.12}{0.26} \right)^2 \approx 0.497$				
$Gini(b[1]:1)$	$= 1 - \left(\frac{1-0.41}{0.74} \right)^2 - \left(\frac{0.5-0.28}{0.74} \right)^2 - \left(\frac{1-0.19}{0.74} \right)^2 - \left(\frac{0.0.12}{0.74} \right)^2 \approx 0.591$				
$Gini(b[1])$	$\approx 0.26 \cdot 0.497 + 0.74 \cdot 0.591 \approx 0.567$				
$Gini(a)$	0.472	0.703	0.567	0.599	
Entropy:	$IG'(a) = - \sum_{v \in V_a} p(v) \cdot \sum_{c \in C} (p_v(c) \cdot \log_2(p_v(c)))$				
$IG'(b[1]:0)$	$= - \frac{0.0.28}{0.26} \cdot \log_2\left(\frac{0.5-0.28}{0.26}\right) - \frac{1-0.12}{0.26} \cdot \log_2\left(\frac{1-0.12}{0.26}\right) \approx 0.998$				
$IG'(b[1]:1)$	$= - \frac{1-0.41}{0.74} \cdot \log_2\left(\frac{1-0.41}{0.74}\right) - \frac{0.5-0.28}{0.74} \cdot \log_2\left(\frac{0.5-0.28}{0.74}\right) - \frac{1-0.19}{0.74} \cdot \log_2\left(\frac{1-0.19}{0.74}\right) \approx 1.43$				
$IG'(b[1])$	$\approx 0.26 \cdot 0.998 + 0.74 \cdot 1.43 \approx 1.32$				
$IG'(a)$	1.19	1.86	1.32	1.55	
Info Gain Ratio:	$H(X) = - \sum_{x \in X} (p(x) \cdot \log_2(p(x)))$				
$SplitInfo(a)$	$= - \sum_{v \in V_a} (p(v) \cdot \log_2(p(v)))$, $IGR(X,a) = \frac{H(X) - IG'(a)}{SplitInfo(a)}$				
$H(X)$	$= -0.41 \cdot \log_2(0.41) - 0.28 \cdot \log_2(0.28) - 0.19 \cdot \log_2(0.19) - 0.12 \cdot \log_2(0.12) \approx 1.86$				
$SplitInfo(b[1])$	$= -0.26 \cdot \log_2(p(0.26)) - 0.74 \cdot \log_2(p(0.74)) \approx 0.827$				
$IGR(b[1])$	$\approx \frac{1.86 - 1.32}{0.827} \approx 0.661$				
$IGR(a)$	0.693	0	0.661	0.355	

Table III. The table is divided into three parts. The first part shows four encodings that entail both defined bits and conditions. The second part contains the contingency table using bits as predictors. The class probability is multiplied with values between 0 and 1, where 0 indicates an unmatched, 1 a full match, and all other values a partial match of the corresponding probability. Total values are calculated per bit (predictor, independent variable, column) and encoding label (class, dependent variable, row). In the third part, the four metrics chi-squared, gini index, entropy and information gain ratio are calculated. An example is given per metric to illustrate the calculation. The shaded row shows the metric calculation results for all four bits (predictors). The contingency table for predicates is calculated similarly.

$$E(v, c) = p(c) \cdot incidence(v) = p(c) \cdot \sum_{c \in C} incidence(v, c)$$

$$\chi^2(v, c) = \frac{(incidence(v, c) - E(v, c))^2}{E(v, c)}$$

$$\chi^2(a) = \sum_{v \in V_a} \sum_{c \in C} \chi^2(v, c)$$

We can formulate χ^2 in terms of the probability instead of the incidence, where $p(v)$ is the child node probability relative to the parent, $p(c)$ is the probability of the class within the parent node, and $p_v(c)$ the probability of the class within the child node:

$$\chi^2(v, c) = \left(\sum_{c \in C} incidence(v, c) \right) \cdot \frac{(p_v(c) - p(c))^2}{p(c)}$$

In the context of decoders, we want a metric that is independent of the number of samples. To achieve this, we can calculate an

alternate χ'^2 where we replace the first factor with the probability $p(v)$. Since we are only interested in comparing chi-squared for different attributes, the change in absolute value is of no relevance since the chi-squared ratios of the attributes remain the same:

$$\chi'^2(a) = \sum_{v \in V_a} p(v) \cdot \sum_{c \in C} \left(\frac{(p_v(c) - p(c))^2}{p(c)} \right)$$

b) *Gini Index*: The Gini index is another classical metric, used e.g. in [20]. The index is calculated per predictor as follows:

$$Gini(v) = 1 - \sum_{c \in C} p_v(c)^2$$

$$Gini(a) = \sum_{v \in V_a} (p(v) \cdot Gini(v)) = \sum_{v \in V_a} p(v) \cdot \left(1 - \sum_{c \in C} p_v(c)^2 \right)$$

The contingency values p are calculated per predictor and class as discussed and the square is summed over all encodings in the encoding set E . Predictors with lower Gini index values are considered better candidates.

c) *Entropy*: The entropy, as used e.g. in [21] or the information gain of a given encoding subset X is calculated as follows:

$$H(X) = - \sum_{x \in X} (p(x) \cdot \log_2(p(x)))$$

The algorithm seeks to maximize the splitting information gain IG , which is the original parent entropy $H(X)$ minus the sum of the resulting entropies of the subsets $H(V_a)$ after splitting on predictor a . The child entropies are modulated with the probability of the child branch relative to all encodings in the current subset:

$$IG(X, a) = H(X) - \sum_{v \in V_a} (p(v) \cdot H(V_a))$$

Equivalently, it suffices to minimize the sum of the modulated child nodes' entropies:

$$IG'(a) = \sum_{v \in V_a} (p(v) \cdot H(V_a)) = - \sum_{v \in V_a} p(v) \cdot \sum_{c \in C} (p_v(c) \cdot \log_2(p_v(c)))$$

d) *Information Gain Ratio*: The information gain ratio, as used e.g. in [22], tries to balance the information gain's bias towards higher child branch numbers by using gain ratio normalization:

$$SplitInfo(a) = - \sum_{v \in V_a} (p(v) \cdot \log_2(p(v)))$$

$$IGR(X, a) = \frac{IG(X, a)}{SplitInfo(a)} = \frac{1}{SplitInfo(a)} \cdot (H(X) - \sum_{v \in V_a} (p(v) \cdot H(V_a)))$$

G. Selecting and Combining Complex Decision Functions

As mentioned, the function $SelectFunction(node, E)$ is where the four criteria of Section IV-F are implemented. All four are applied as follows:

- The set of all predictors is constructed. This includes all bits and all predicates defined in any encoding in the current subset. This set is filtered to exclude all bits and predicates that have already been tested. Note that predictors are not whole clauses, but single predicates therein. This offers more flexibility and can be the only solution in some situations.
- The appropriate metric is calculated for each set element.
- Decision function selection offers two variants: In *single mode*, the predictor, either bit or predicate, with the best metric is

	X*2 Multi	Gini Idx Multi	Entropy Multi	I-Gain Ratio Multi	X*2 Single	Gini Idx Single	Entropy Single	I-Gain Ratio Single
SPARC v8	Instructions	503	494	1218	214	560	575	1112
	Total Nodes	2.051	1.965	2.499	2.37	10.028	9.864	9.827
	Average Static Depth	0.96	0.997	1.291	1.713	0.35	0.344	0.406
	Generation Time [s]	56	55	58	66	56	57	58
	Gen. Peak Mem. [MB]	56	55	58	66	56	57	58
MIPS2 v6.05	Instructions	980	980	3048	14426	583	591	1642
	Total Nodes	2.545	2.545	2.493	2.392	10.962	10.568	10.073
	Average Static Depth	31.653	30.322	53	219	48.857	52.759	60.756
	Generation Time [s]	184	184	184	207	186	184	184
	Gen. Peak Mem. [MB]	184	184	184	207	186	184	184
ARMv7-A	Instructions	8008	8008	8008	32930	837	828	1539
	Total Nodes	86.098	85.727	86.614	103.252	172.705	174.085	125.151
	Generation Time [s]	375	393	376	435	371	376	375
	Gen. Peak Mem. [MB]	375	393	376	435	371	376	375
	Gen. Peak Mem. [MB]	375	393	376	435	371	376	375

Fig. 3: Decoder generation statistics.

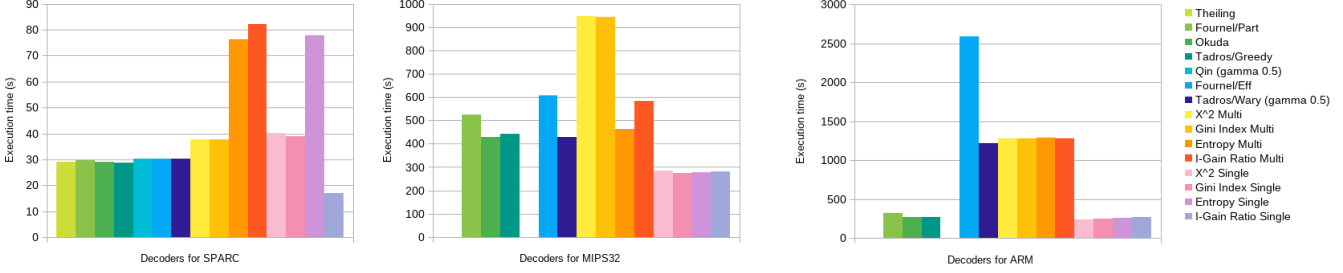


Fig. 4: Decoder execution statistics.

chosen, where ‘best’ implies maximum or minimum depending on the algorithm. *Multiple mode* also chooses the best single predicate, but handles bits differently: These are first sorted according to their merit. The next best bit is then added to the candidate mask and the encodings split accordingly. As the mask now contains two bits, the result is at most a four-branch tree, for which the metric is again calculated. This process continues iteratively with the next best bit as long as the metric improves and the mask length is within 8 bits to prevent overly broad trees. The resulting J -bit mask is then compared to the best single predicate and the better of the two chosen.

- We refrain from any pruning or postprocessing for better contrast between the different metrics.

H. Decoder Generation

V. EXPERIMENTAL RESULTS

In order to implement our algorithm, we largely rewrote the open-source processor generation tool TRAP-Gen (TRAnsactional Automatic Processor GENerator) [23] for our purposes. The tool could initially generate a SPARC SystemC processor model from a high-level ISA description in Python 2.7 using Qin’s algorithm [5]. Neither MIPS32 nor ARMv7 could be generated using this setup. We thus implemented our algorithm using all four metrics in both single and multiple mode and used it to generate eight decoders for each of the three platforms. Generation statistics are shown in Fig. 3. The three platforms are ordered according to rising complexity, as can be seen from the number of instructions, total number of tree nodes, generation time and peak memory during generation. Single mode generally creates deeper trees (larger “Average static depth”) than multiple mode, which, in turn, makes it up in tree breadth (similar “Total Nodes”). As will be obvious in the execution statistics, tree depth is only one of several factors that determine the execution speed.

A. Decoder Execution

As we posit that the runtime decoding speed is the most significant decoder evaluation criteria, we proceeded to measure the execution performance of each decoder. To this end, we first tested all generated decoders for functional correctness, i.e. we ascertained that all instructions are decoded properly. Then, we used the GNU toolchains with no optimization to cross-compile the *mini* version of PolyBench [24], a suite of 30 numerical benchmarks in various application domains such as linear algebra, image processing, physics, dynamic programming, statistics, and others. All 30 benchmarks were then run on QEMU [13] in user mode in combination with the toolchain’s GDB, which dumped the instruction machine code to a text file. Next, a decoder test mode was coded in TRAP-Gen, where, instead of executing binaries, a text file is read and the hex code loaded into the memory model, which, in the functional abstraction level used in test mode, is a simple array. The instruction behavior is ignored in test mode, meaning that the instruction is only read from memory, decoded, then discarded. This avoids distorting the results by the overhead of executing the instruction behavior. Code for monitoring the time was also inserted in the decoder C++ implementation just

before and immediately after decoding each instruction and summed up for all instructions of a benchmark.

To compare against existing solutions, we also implemented seven other algorithms as published by Theiling [1], Qin [5], Fournel (“Part” and “Eff”) [3], Okuda [4] and Tadros (“Greedy” and “Wary”) [10]. As neither Theiling nor Qin can handle propositions, they were only applicable to SPARC. Although Fournel and Okuda both attempt to handle propositions, they were either very resource-intensive (Fournel/Part), required hardcoding some instructions (Fournel/Eff), or produced functionally erroneous decoders (Okuda). To be able to roughly compare the performance nevertheless, we followed the approach delineated in [6] to adapt the algorithms so as to produce functionally correct decoders for our chosen platforms. The results are shown in Fig. 4. We grouped and color-coded the algorithms as follows: The first group (green) is composed of greedy algorithms that do not attempt to optimize the tree cost, the second (blue) performs some kind of optimization, the third (yellow to red) is our multiple mode and the fourth (pastel) our single mode. Interestingly, the optimization group generally performs worse than greedy algorithms. This is especially apparent in the results for the ARM platform. This would be partially the consequence of the misleading, heuristic cost model, which was analyzed in depth in [7]. Regarding our algorithms, single mode performed consistently and by a large margin better than multiple mode. As the one makes up in breadth what the other does in depth, we suspect this is due to the cost impact of tree breadth as described in [7]. It warrants implementing said cost model to our platform to better explain the results. Altogether our algorithms are the second published solution that produces *functionally correct* decoders, together with [10]. As to performance, our algorithms are among the fastest, only slightly beaten for the SPARC platform. Regarding the individual metrics, they produce very similar results for single predictors. We attribute this to the fact that, since only a single bit or predicate is chosen, there is a high likelihood of all four algorithms selecting the same predictor. Multiple predictors obviously offer more space for divergence, with Entropy/Information Gain Ratio performing worse with SPARC but much better with MIPS, compared to Chi-squared and Gini Index.

To conclude, we note the difference in decoding speed between ISAs using propositions (MIPS and ARM) and those doing without them (SPARC), as apparent in the execution time scale. This is a further indication of the impact of ISA complexity on decoder performance. As this trend is expected to continue, the necessity of optimized decoder generation algorithms is further emphasized.

VI. CONCLUSION

We have conceived and implemented an algorithm that solves the problem of generating instruction decoders for complex and irregular instruction sets. We benchmarked the generated decoders to measure the execution speed. From a functional point of view, only one other published algorithm is currently capable of producing fully automated and functionally correct decoders. From a performance perspective, our algorithms outperform existing decoders. Our future work is aimed at improving the decision function selection algorithm in combination with an accurate cost model.

REFERENCES

- [1] H. Theiling, “Generating decision trees for decoding binaries,” in *Proceedings of the ACM SIGPLAN Workshop on Languages, Compilers and Tools for Embedded Systems (LCTES)*, jun 2001, pp. 112–120.
- [2] T. Ratsimbahotra, H. Cassé, and P. Sainrat, “A versatile generator of instruction set simulators and disassemblers,” in *Proceedings of the International Symposium on Performance Evaluation of Computer Telecommunication Systems (SPECTS)*, jul 2009, pp. 65–72.
- [3] N. Fournel, L. Michel, and F. Pétrot, “Automated generation of efficient instruction decoders for instruction set simulators,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, nov 2013, pp. 739–746.
- [4] K. Okuda and H. Takeyama, “Decision tree generation for decoding irregular instructions,” in *Proceedings of the Conference on Design, Automation Test in Europe Conference (DATE)*, mar 2016, pp. 1592–1597.
- [5] W. Qin and S. Malik, “Automated synthesis of efficient binary decoders for retargetable software toolkits,” in *Proceedings of the Design Automation Conference (DAC)*, jun 2003, pp. 764–769.
- [6] L. Tadros, “Functional analysis and performance evaluation of decoder decision tree generation algorithms,” in *Proceedings of the 37th International Conference on Modelling and Simulation (ECMS)*, jun 2023, pp. 262–269. [Online]. Available: <https://www.scs-europe.net/dlib/dl-index.htm>
- [7] —, “A cost model for decoder decision trees,” in *Proceedings of the 2020 European Symposium on Software Engineering (ESSE)*. New York, NY, USA: Association for Computing Machinery, nov 2020, pp. 142–147.
- [8] H. Wagstaff, “From high level architecture descriptions to fast instruction set simulators,” PhD thesis, The University of Edinburgh, nov 2015.
- [9] N. Y. Fokina and M. A. Solovev, “Automated generation of machine instruction decoders,” vol. 45, pp. 390–397, 2019.
- [10] L. Tadros, “Automated generation of decision trees for decoding irregular instruction sets,” in *Communications of the 38th International Conference on Modelling and Simulation (ECMS)*, pp. 459–465. [Online]. Available: <https://www.scs-europe.net/dlib/dl-index.htm>
- [11] Z. Cheng, X. Cao, and J. Zhao, “Research on approaches of implementation of instruction set simulator,” in *Proceedings of the 2021 International Conference on Electronic Information Engineering and Computer Science (EIECS)*, 2021, pp. 1008–1011.
- [12] Free Software Foundation, Inc., “GNU binutils,” <http://www.gnu.org/software/binutils>, version 2.38.
- [13] F. Bellard, “QEMU. Open Source Processor Emulator,” wiki.qemu.org/Main_Page, version 6.2.0.
- [14] Y. Klimiankou, “Rapid instruction decoding for IA-32,” in *Perspectives of System Informatics*, N. Bjørner, I. Virbitskaite, and A. Voronkov, Eds. Cham: Springer International Publishing, 2019, pp. 1–9.
- [15] *The SPARC Architecture Manual*, SPARC International Inc. Std. Version 8, Revision SAV080SI9308, 1992.
- [16] *ARM Architecture Reference Manual*, ARM Ltd. Std. ARMv7-A and ARMv7-R edition (C.b), jul 2012.
- [17] *MIPS Architecture for Programmers. Volume II-A: The MIPS32 Instruction Set Manual*, Imagination Technologies Ltd. Std. MD00086, Revision 6.05, may 2016.
- [18] L. Rokach and O. Maimon, *Data Mining with Decision Trees. Theory and Applications*. River Edge, NJ: World Scientific Publishing Co., Inc., 2015.
- [19] G. V. Kass, “An exploratory technique for investigating large quantities of categorical data,” *Journal of The Royal Statistical Society Series C-applied Statistics*, vol. 29, pp. 119–127, 1980.
- [20] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Chapman & Hall/CRC, 1984.
- [21] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, pp. 81–106, 1986.
- [22] —, *C4.5: Programs for Machine Learning*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1993.
- [23] L. Fossati, “trap-gen,” <https://code.google.com/archive/p/trap-gen/>, version 0.4.5.
- [24] L.-N. Pouchet, “PolyBench/C,” <https://web.cs.ucla.edu/~pouchet/software/polybench/>, version 4.2.1.